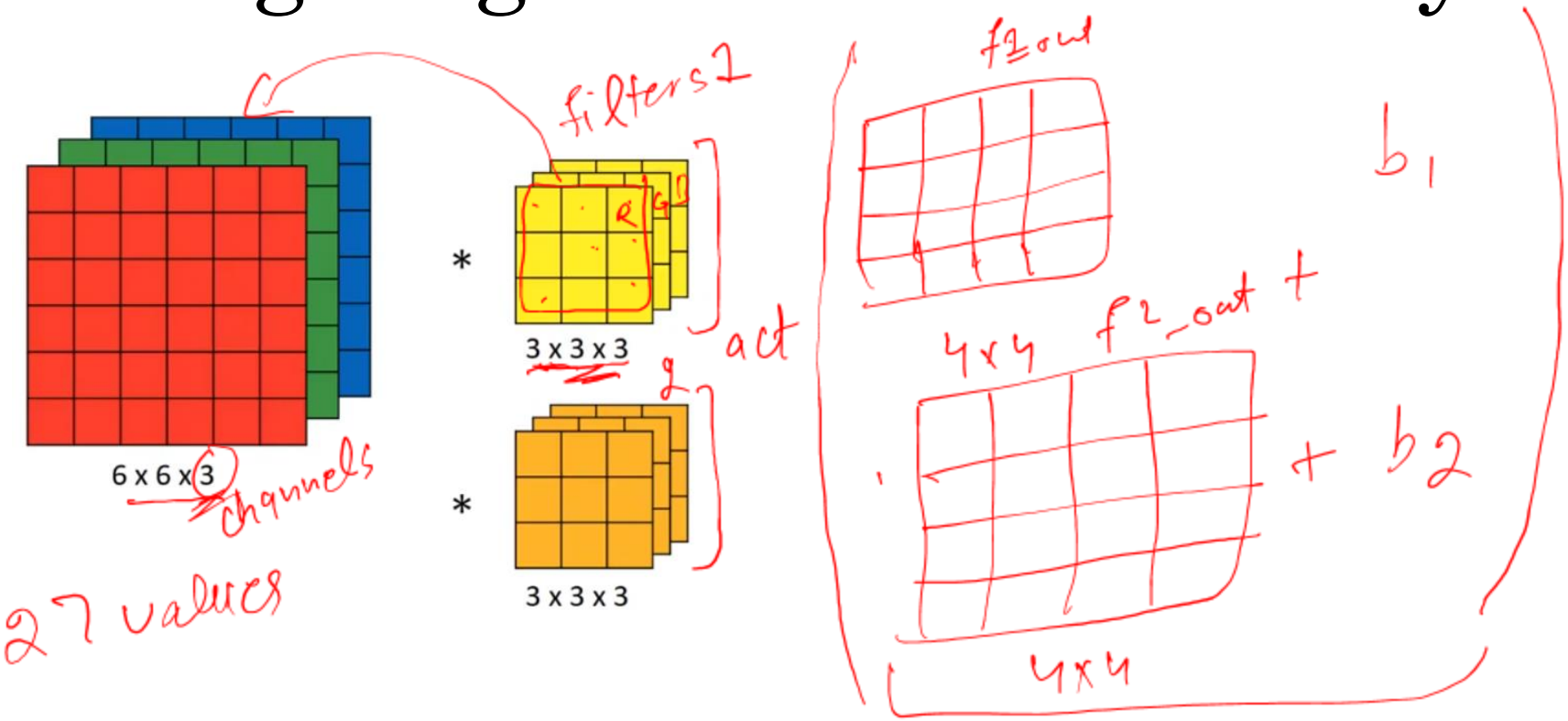
Convolution Neural Networks
Notable Architectures
09 September 2025

Prepared by Tanveer Hussain (Fellow HEA)
Lecturer, Edge Hill University
htanveer3797@gmail.com

Lecture Outline

- Designing a Convolution Layer
- Advantages of using Convolution for Visual Data
- Notable CNN Architectures
- Designing your CNN

Designing a Convolution Layer



nn.Conv2D(in_channels, out_..., hyperparameters)

tf.conv

Conv2D(...)

Image Classification

Why convolutions?

Image Classification

Why convolutions?

- **Parameter sharing:** A feature detector (filter) such as vertical edge detector useful in one part of the image is probably useful in another part of the image.

Image Classification

Why convolutions?

- **Parameter sharing:** A feature detector (filter) such as vertical edge detector useful in one part of the image is probably useful in another part of the image.
- **Sparsity of connections:** In each layer, the output value typically depends on specific small number of inputs.

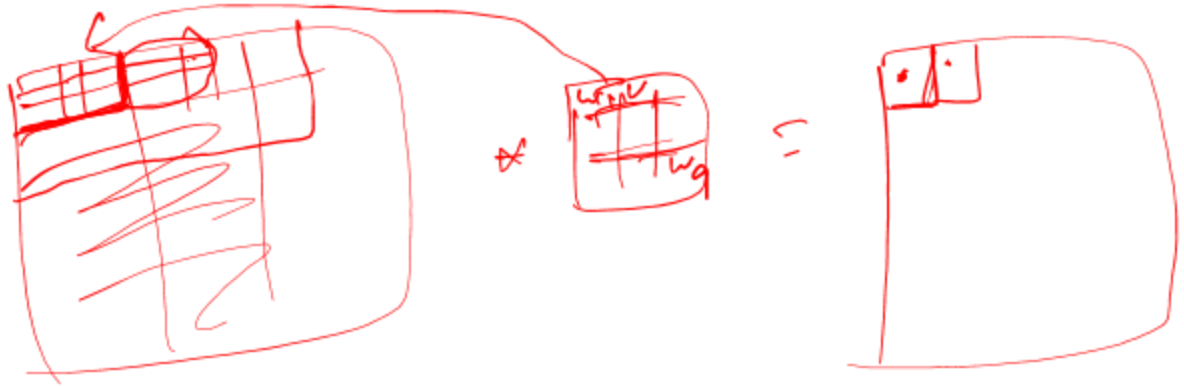
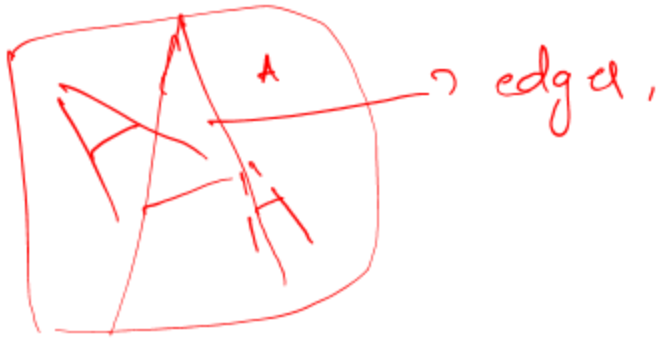


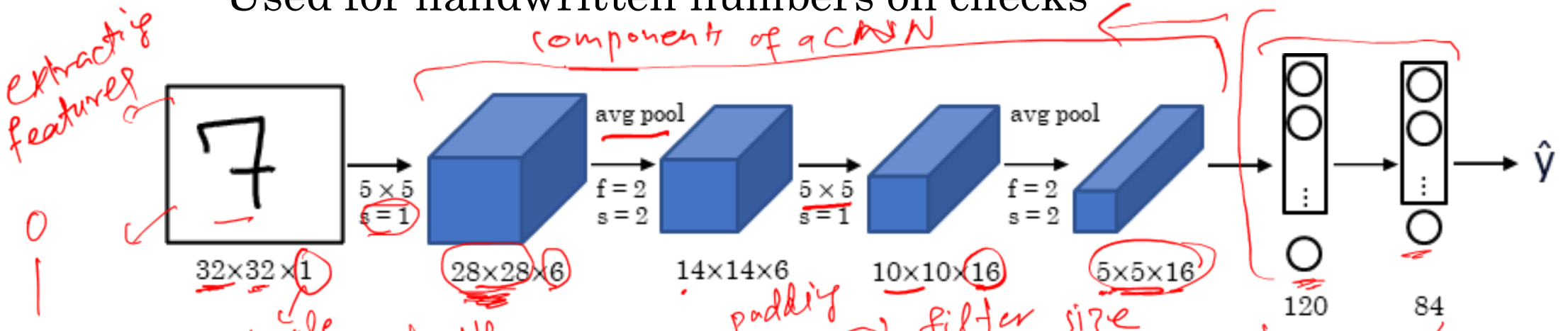
Image Classification

- Image Classification/Visual Recognition model's Expectations
  1. Maintain 2D structure logic
  2. Shift invariant (actually, equivariant)
  3. Consider local correlations first
  4. Hierarchically growing field of view
  5. Hierarchically progressing complexity
  6. Reasonable amount of params



Notable CNN Architectures

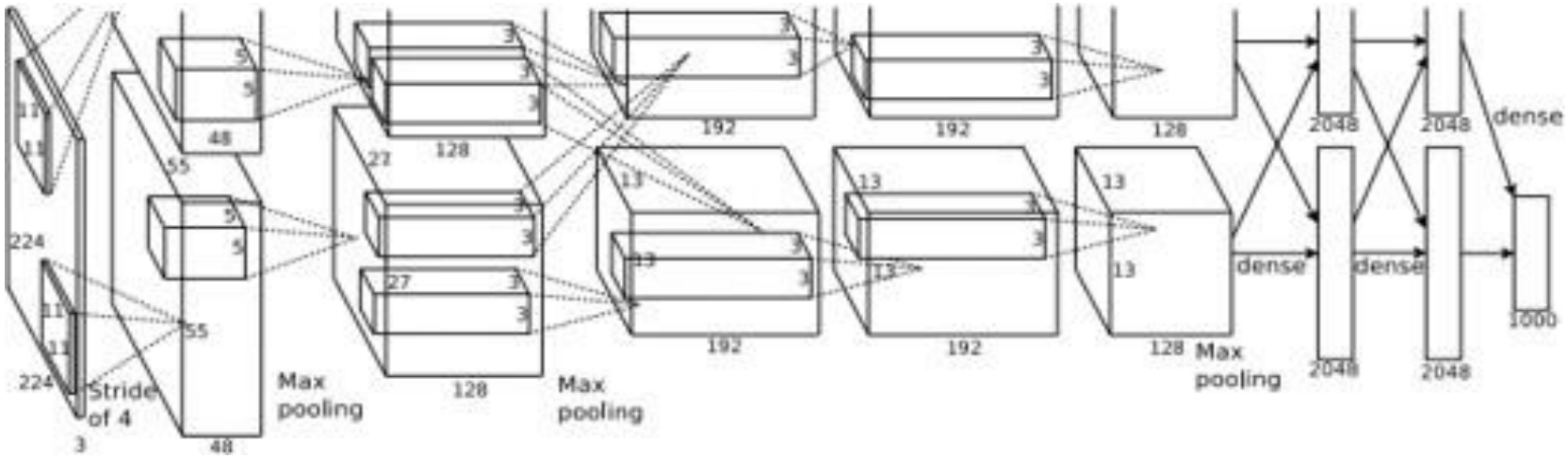
- LeNet-5 (1998)
  - It contains 2 convolutional layers, 2 pooling layers, and 2 fully connected layers
  - Used for handwritten numbers on checks



[PDF] LeNet - Yann LeCun
yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf
by Y LeCun - 1998 - Cited by 17209 - Related articles
OF THE IEEE, NOVEMBER 1998. 1 ... Real-life document recognition systems are composed .... C onvolutional Neural Network c alled L eNet - 5 des c rib ed in.

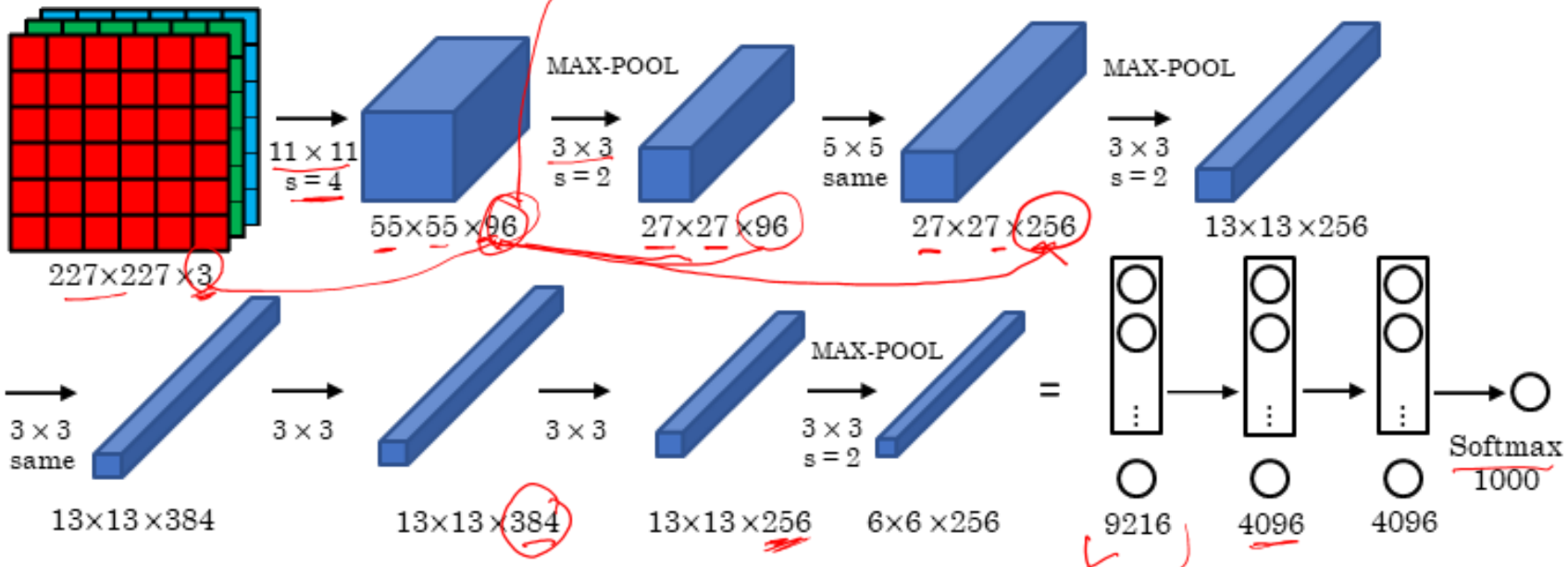
Notable CNN Architectures

- AlexNet (2012)
  - First big improvement in image classification
  - Made use of CNN, pooling, dropout, ReLU and training on GPUs.
  - It contains 5 convolutional layers, followed by max-pooling layers; with three fully connected layers at the end.



Notable CNN Architectures

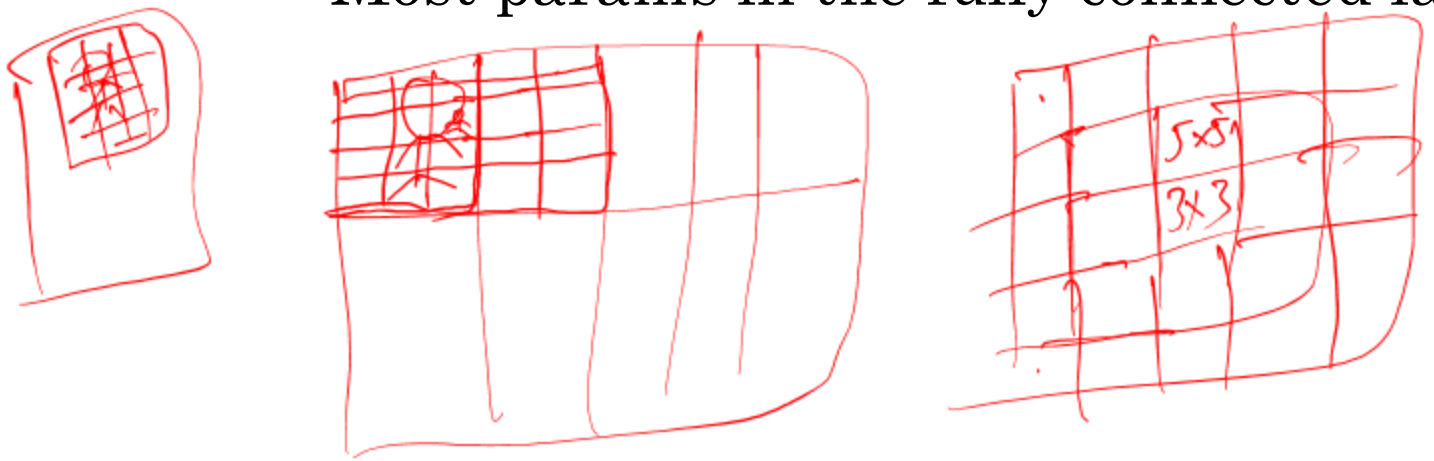
- AlexNet (2012)



[Krizhevsky et al., 2012. ImageNet classification with deep convolutional neural networks]

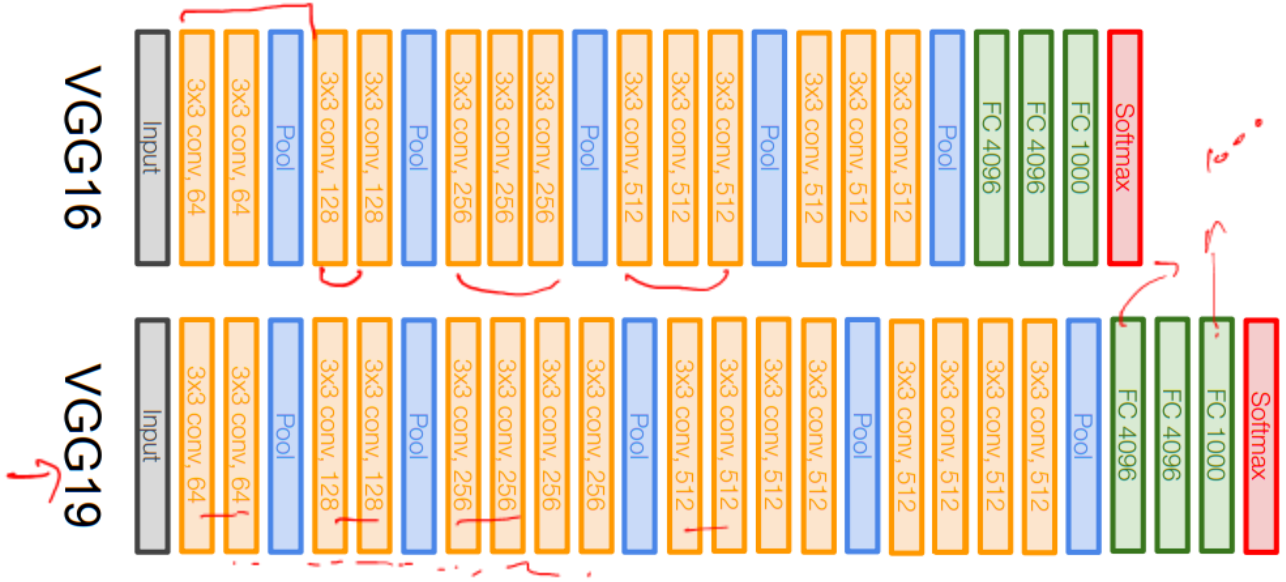
Notable CNN Architectures

- VGGNet
  - Stack of three 3x3 conv (stride 1) layers has same effective receptive field as one 7x7 conv layer
  - But deeper, more non-linearities and lesser parameters
  - It has 13 or 16 conv layers with 3 fully-connected layers. Most params in the fully connected layer



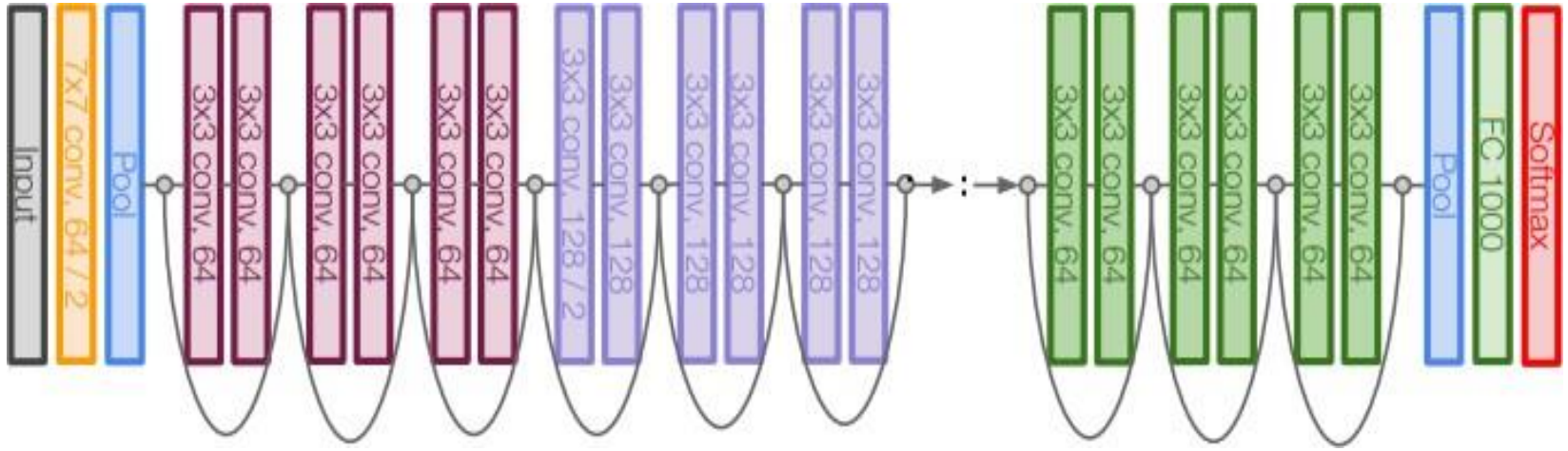
Notable CNN Architectures

- VGGNet
  - Stack of three 3x3 conv (stride 1) layers has same effective receptive field as one 7x7 conv layer
  - But deeper, more non-linearities and lesser parameters
  - It has 13 or 16 conv layers with 3 fully-connected layers. Most params in the fully connected layer



Notable CNN Architectures

- ResNet
  - Very deep networks using residual connections
  - 152-layer model for ImageNet
  - Stacked Residual Blocks

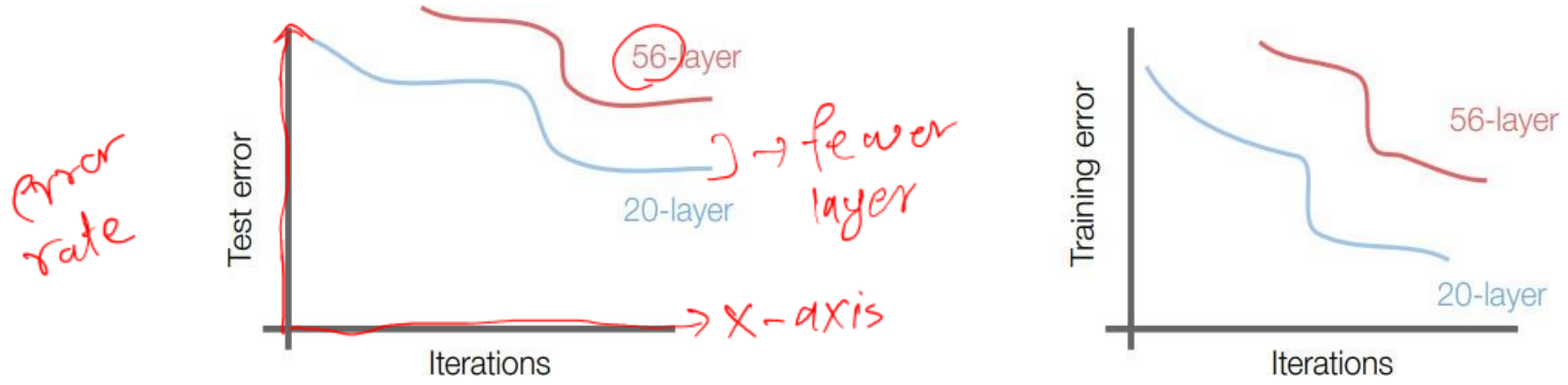


Notable CNN Architectures

- ResNet
  - What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

Notable CNN Architectures

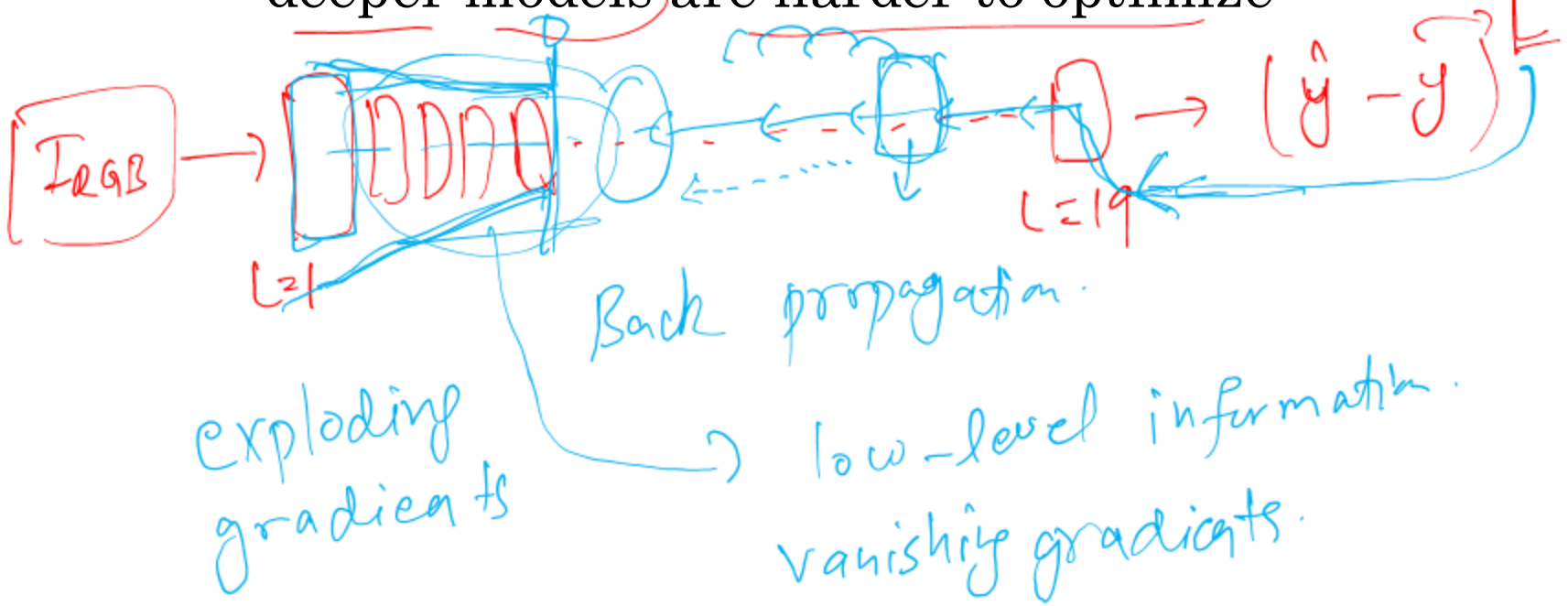
- ResNet
  - What happens when we continue stacking deeper layers on a "plain" convolutional neural network?



  - A 56-layer model performs worse on both test and training error.
  - The deeper model performs worse, but it's not caused by overfitting!

Notable CNN Architectures

- ResNet
  - **Fact:** Deep models have more representation power (more parameters) than shallower models.
  - **Hypothesis:** The problem is an optimization problem, deeper models are harder to optimize

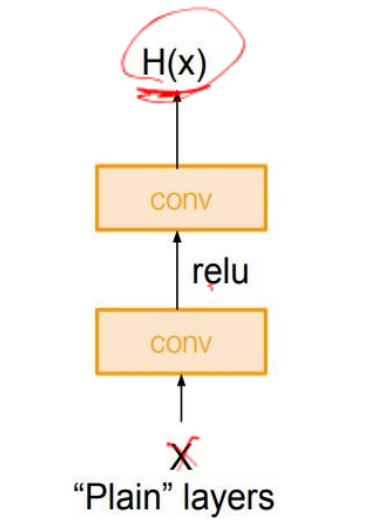


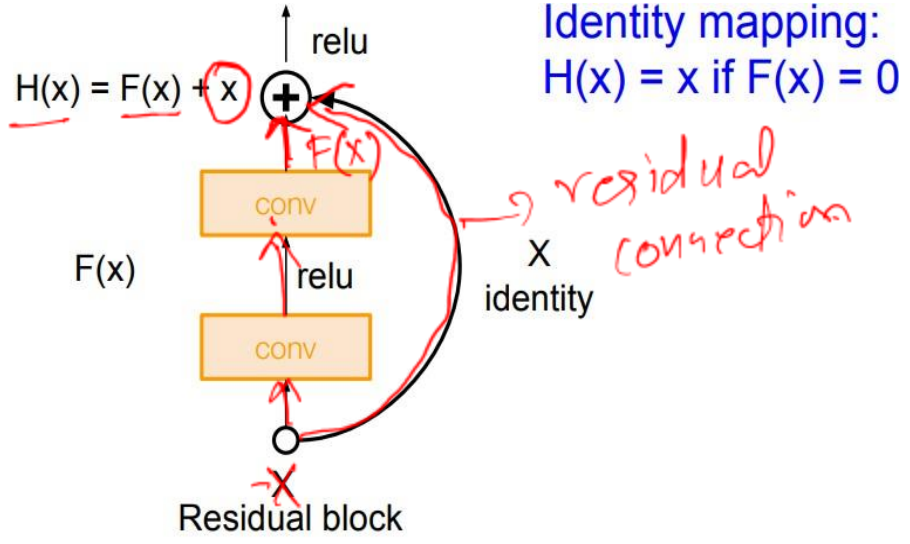
Notable CNN Architectures

- ResNet
  - **Fact:** Deep models have more representation power (more parameters) than shallower models.
  - **Hypothesis:** The problem is an optimization problem, deeper models are harder to optimize.
  - **Solution:** Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

Notable CNN Architectures

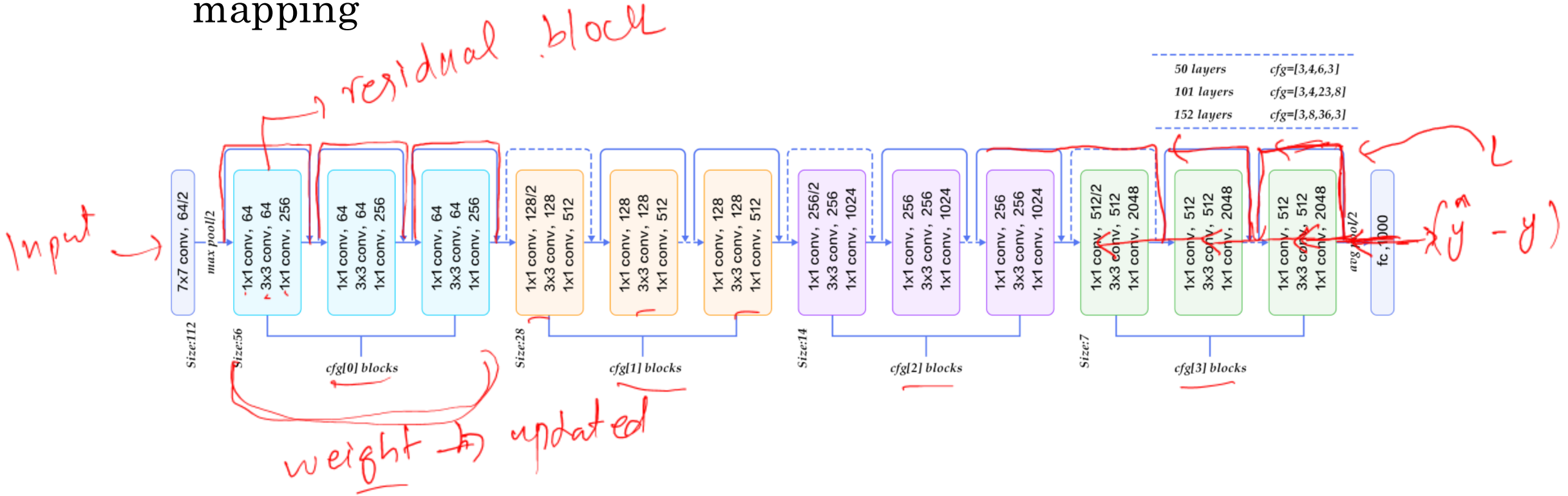
- ResNet
  - **Solution:** Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping





Notable CNN Architectures

- ResNet
  - **Solution:** Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

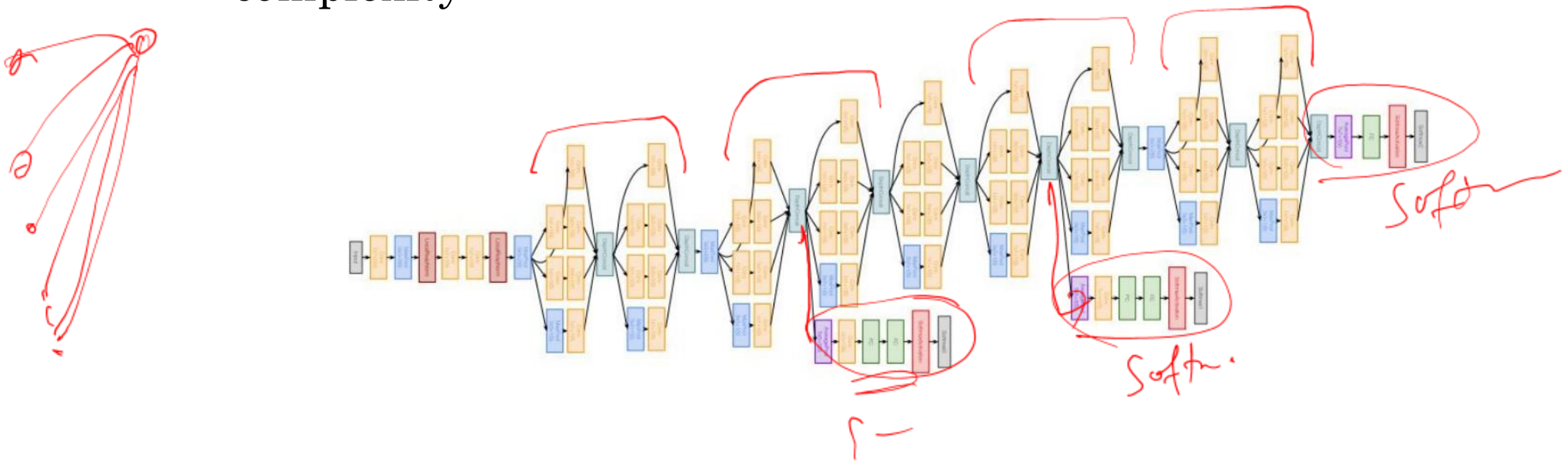


Notable CNN Architectures

- InceptionNet
  - Going Deep: 22 layers
  - Only 5 million parameters! (12x less than AlexNet and 27x less than VGGNet)
  - Introduced ”Inception module”
  - Introduced ”bottleneck” layers that use 1x1 convolutions to reduce feature channel size and computational complexity

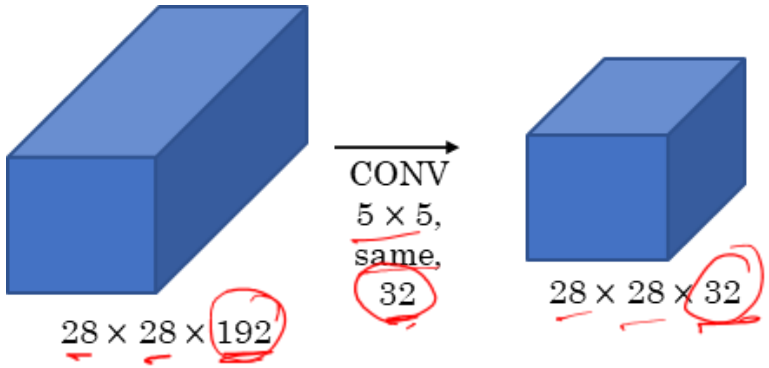
Notable CNN Architectures

- InceptionNet
  - Introduced ”Inception module”
  - Introduced ”bottleneck” layers that use 1x1 convolutions to reduce feature channel size and computational complexity



Notable CNN Architectures

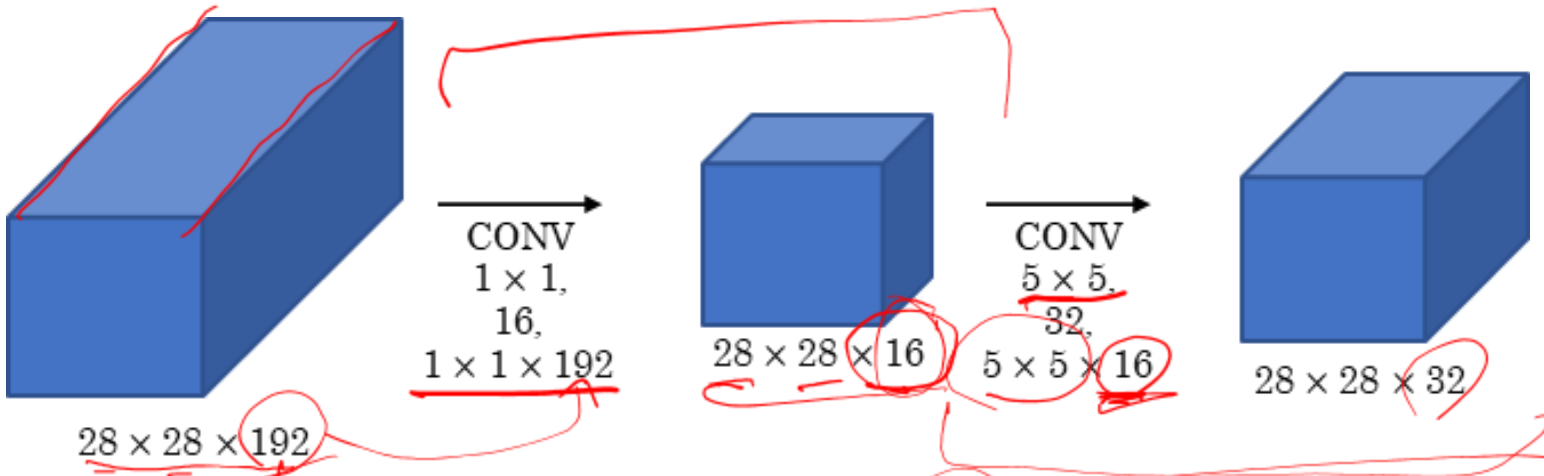
- The problem of computational cost



$5 \times 5 \times 192 \times 32 = 153,600$

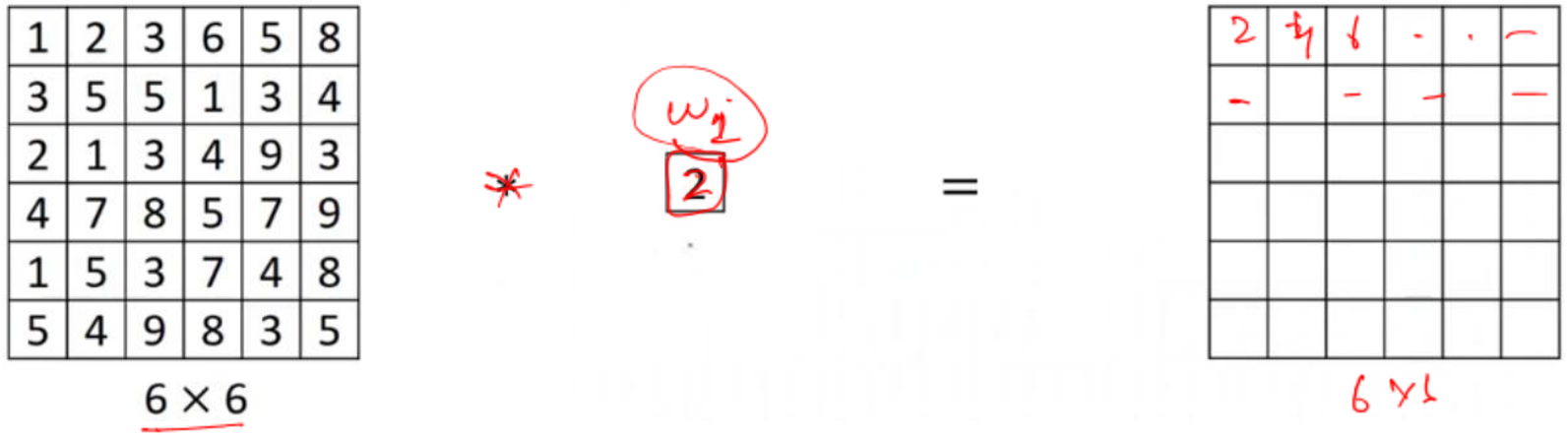
Notable CNN Architectures

- The problem of computational cost



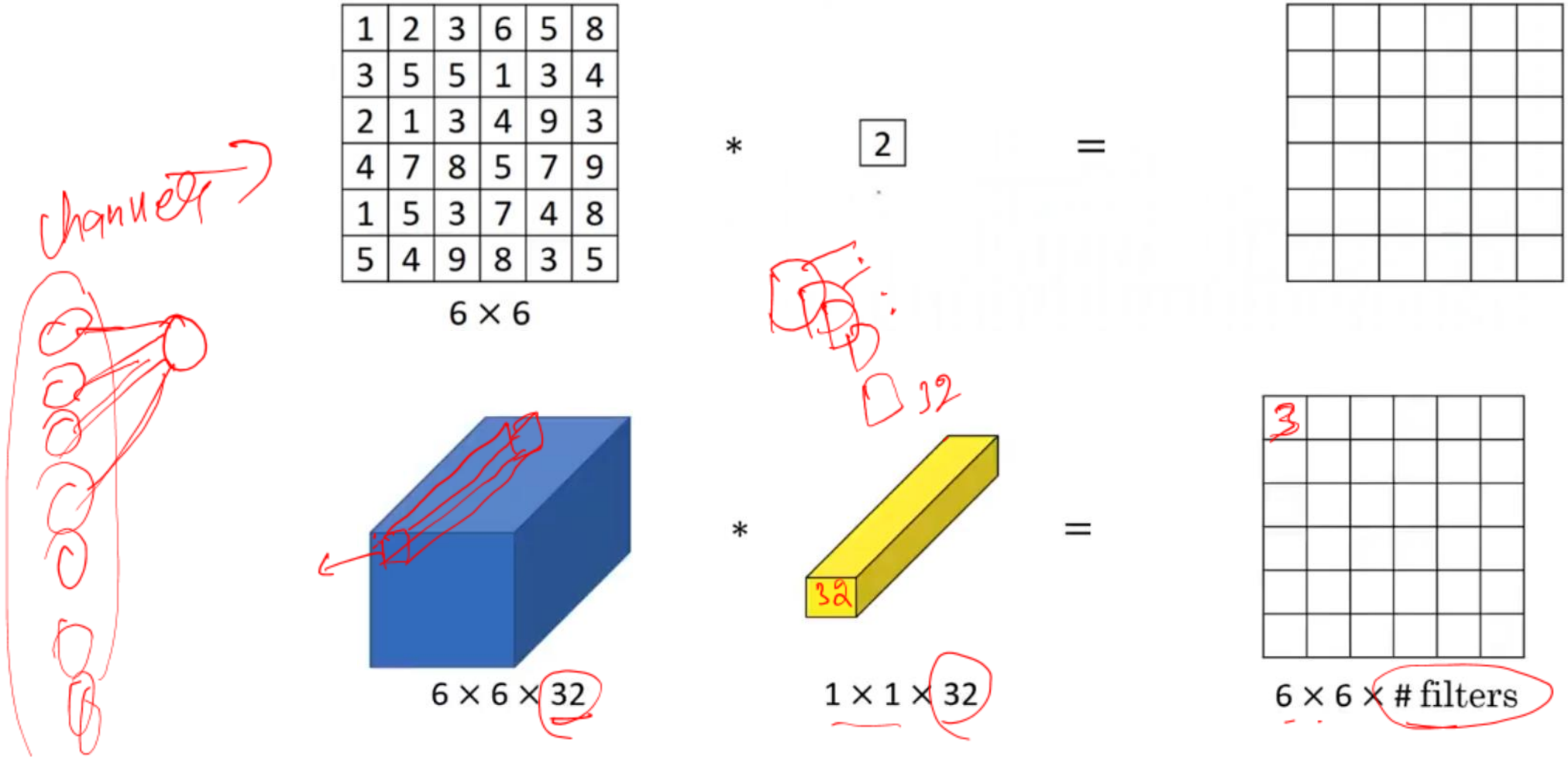
Notable CNN Architectures

- Is it useful during training?



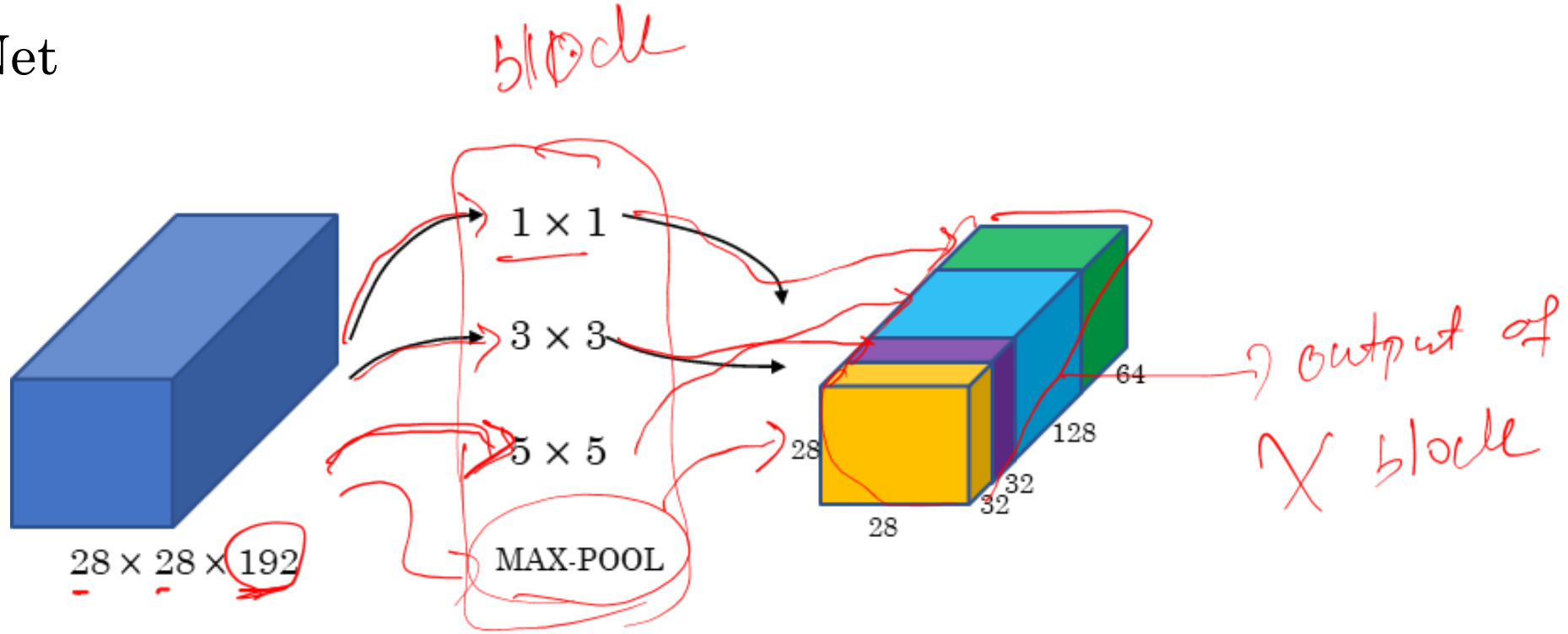
Notable CNN Architectures

- Is it useful during training?



Notable CNN Architectures

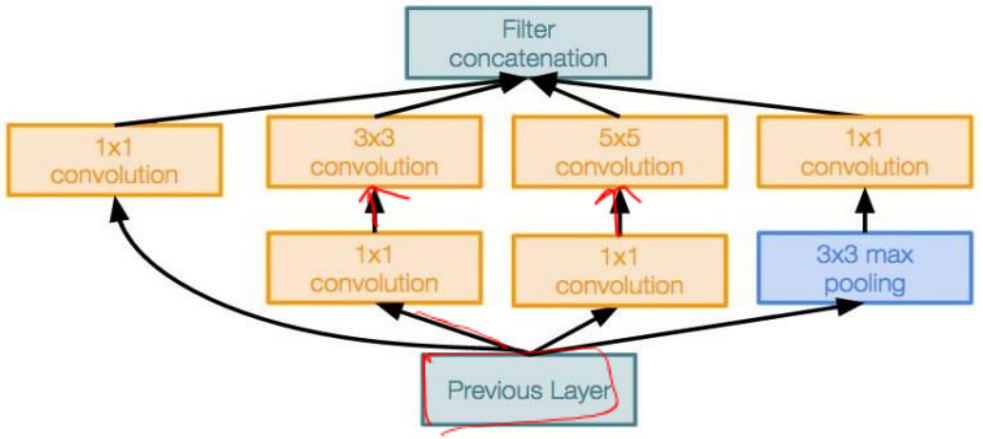
- InceptionNet



Notable CNN Architectures

- **Inception module:** design a good local network topology (network within a network) and then stack these modules on top of each other

another paper



Inception module

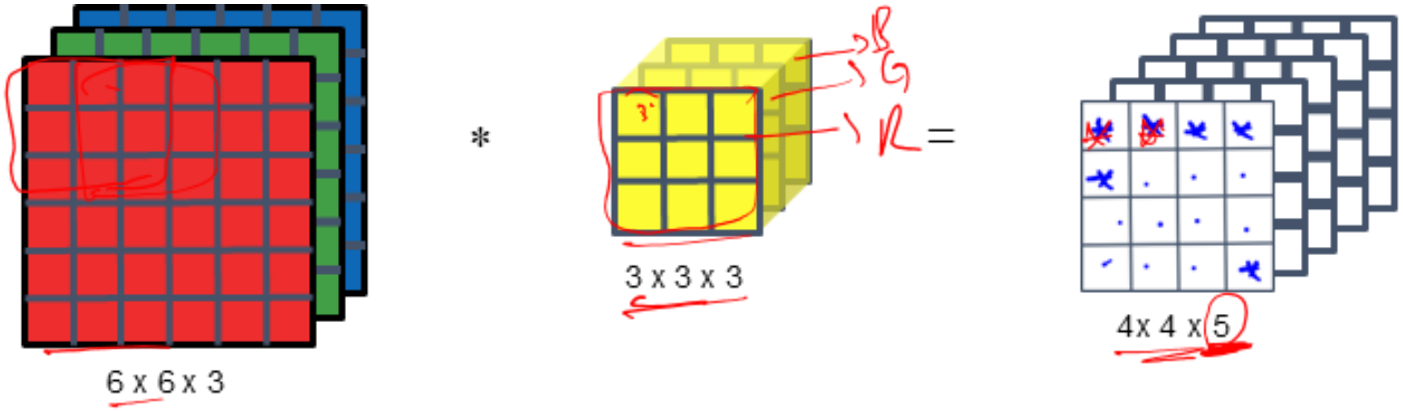
Notable CNN Architectures

- Motivation for MobileNets
  - Low computational cost at deployment
  - Useful for mobile and embedded vision applications
  - **Key idea:** Normal vs. depthwise separable convolutions

MACs
↓
multip/
addit.

Notable CNN Architectures

- Normal convolution



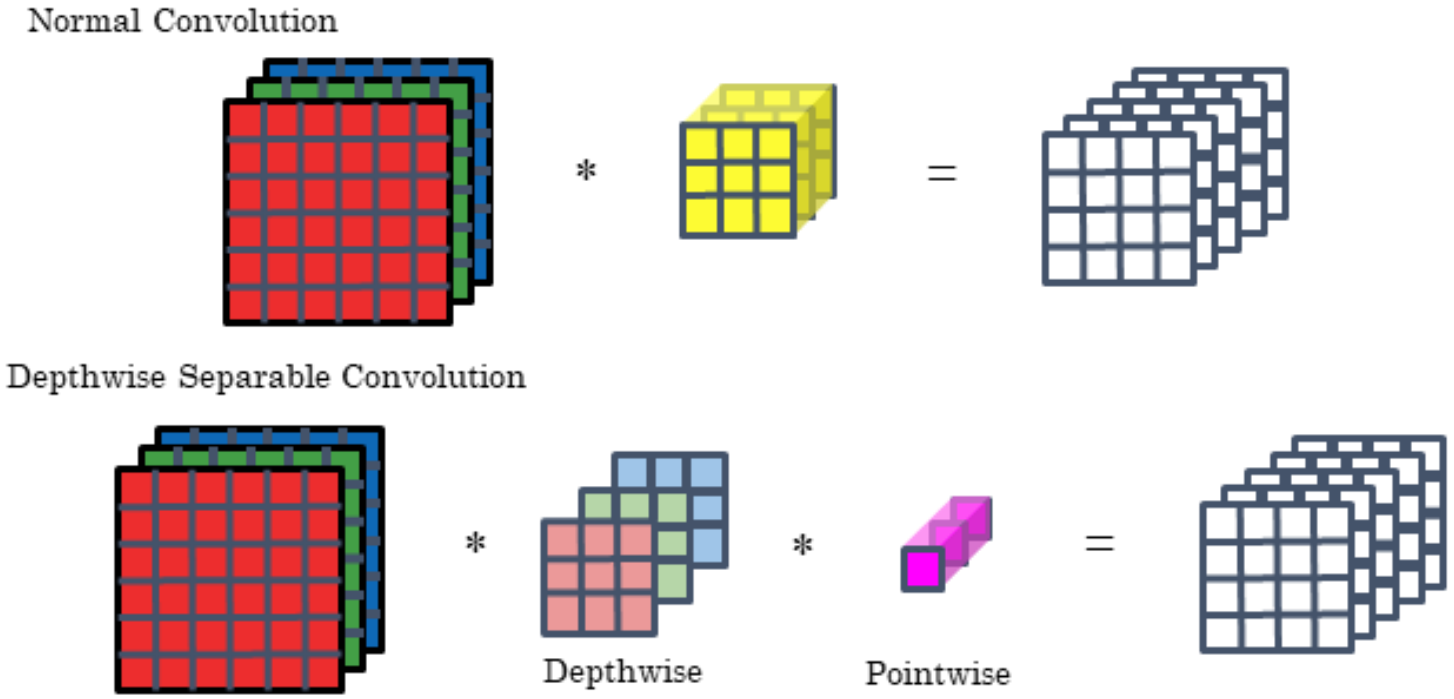
Computational cost = #filter params x # filter positions x # of filters

2160 = 3x3x3 x 4x4 x 5

27 x 16 x 5

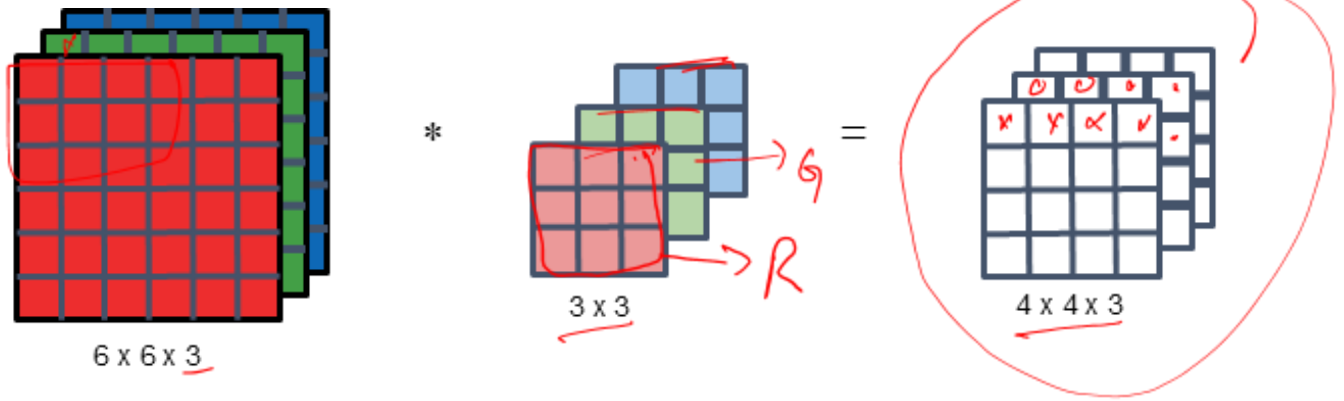
Notable CNN Architectures

- Normal vs depthwise separable convolution



Notable CNN Architectures

- Depthwise convolution

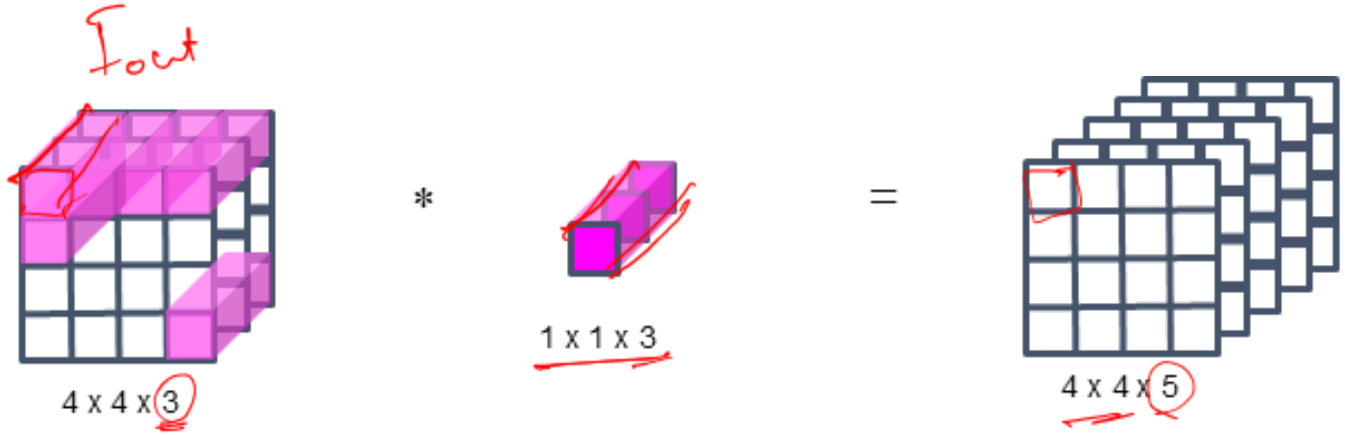


Computational cost = #filter params x # filter positions x # of filters

$$432 = \frac{3\times 3}{6} \times \frac{4\times 4}{16} \times \frac{3}{3}$$

Notable CNN Architectures

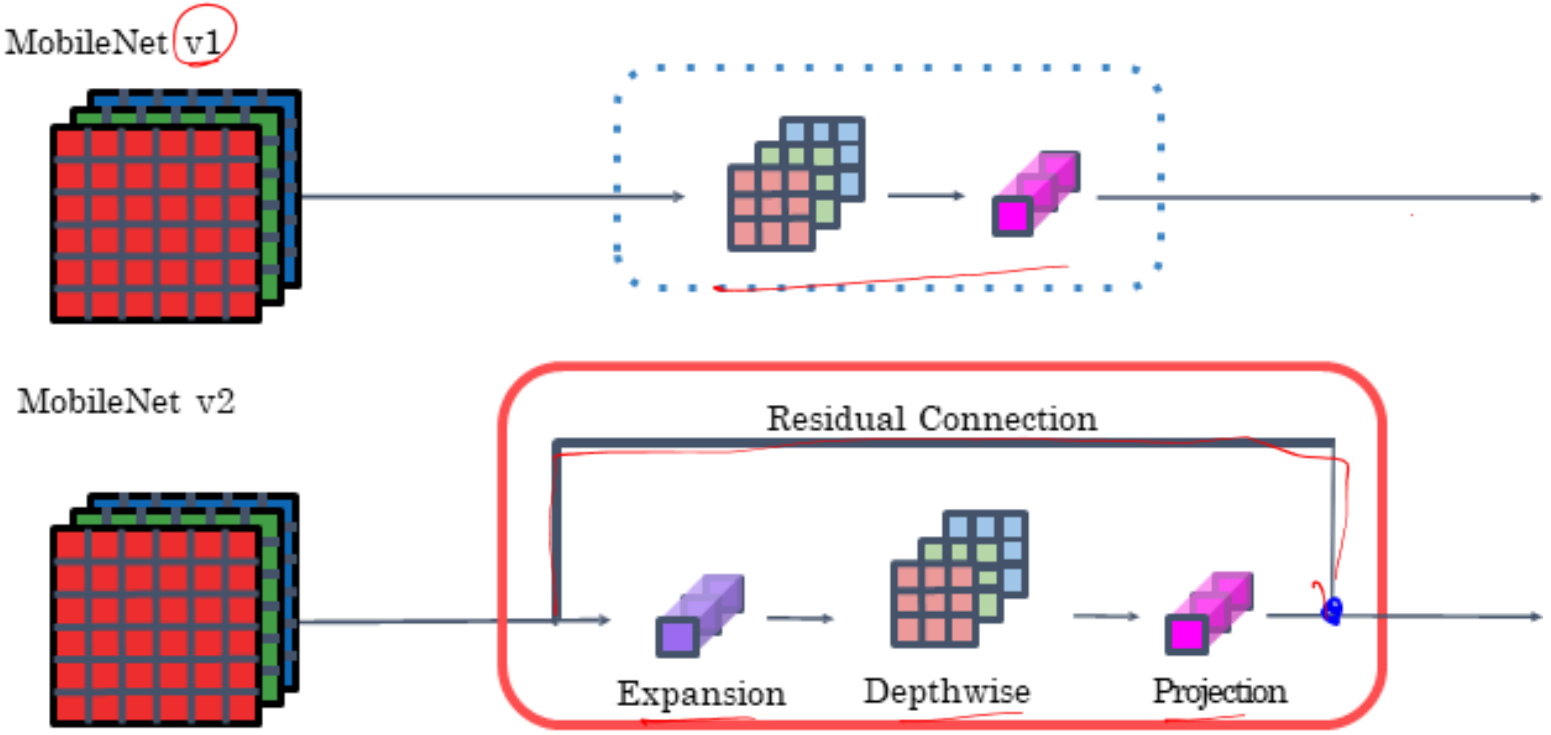
- Pointwise convolution



Computational cost = #filter params x # filter positions x # of filters

Notable CNN Architectures

- MobileNet



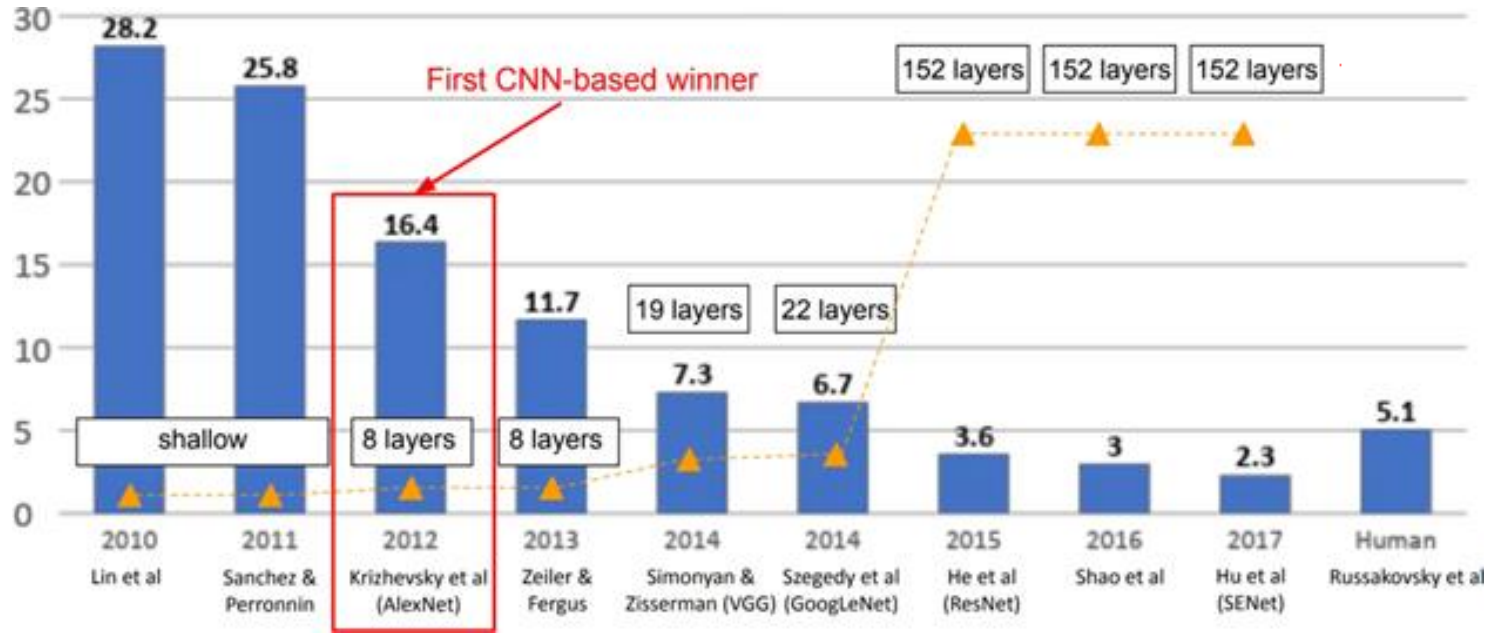
Designing Data-Specific CNN Model

Image Classification Dataset

- ImageNet
  - The most extensive data for Image Classification
  - 3 RGB channels from 0 to 255
  - 14,197,122 images
  - 1000 classes

Image Classification Dataset

- ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



Transfer Learning

- Improvement of learning in a new task through the transfer of knowledge from a related task that has already been learned.
- We will look at one strategy of transfer learning called Fine-Tuning

When to Fine-tune?

1. New dataset is small with distribution similar to original dataset.
   - Keep the feature extraction part fixed and fine-tune the classifier part of the network

When to Fine-tune?

1. New dataset is small with distribution similar to original dataset.
   - Keep the feature extraction part fixed and fine-tune the classifier part of the network
2. New dataset is large with similar distribution to the original dataset
   - Fine tune both the feature extractor and the classifier part of the network

When to Fine-tune?

1. New dataset is small with distribution similar to original dataset.
   - Keep the feature extraction part fixed and fine-tune the classifier part of the network
2. New dataset is large with similar distribution to the original dataset
   - Fine tune both the feature extractor and the classifier part of the network
3. New dataset is small but different distribution from the original dataset
   - Use SVM classifier on the features extracted from the feature extractor part of the Network

When to Fine-tune?

1. New dataset is small with distribution similar to original dataset.
   - Keep the feature extraction part fixed and fine-tune the classifier part of the network
2. New dataset is large with similar distribution to the original dataset
   - Fine tune both the feature extractor and the classifier part of the network
3. New dataset is small but different distribution from the original dataset
   - Use SVM classifier on the features extracted from the feature extractor part of the Network
4. New dataset is large and different distribution from the original dataset
   - Fine tune both the feature extractor and the classifier part of the network

References

- https://web.eecs.umich.edu/~justincj/slides/eecs498/WI2022/598_WI2022_lecture02.pdf
- **https://web.eecs.umich.edu/~justincj/slides/eecs498/WI2022/598_WI2022_lecture03.pdf**
- **https://towardsdatascience.com/training-deep-neural-networks-9fdb1964b964**
- **https://www.charles-deledalle.fr/pages/teaching.php**
- **https://www.charles-deledalle.fr/pages/files/ucsd_ece285_mlip/3_deep.pdf**